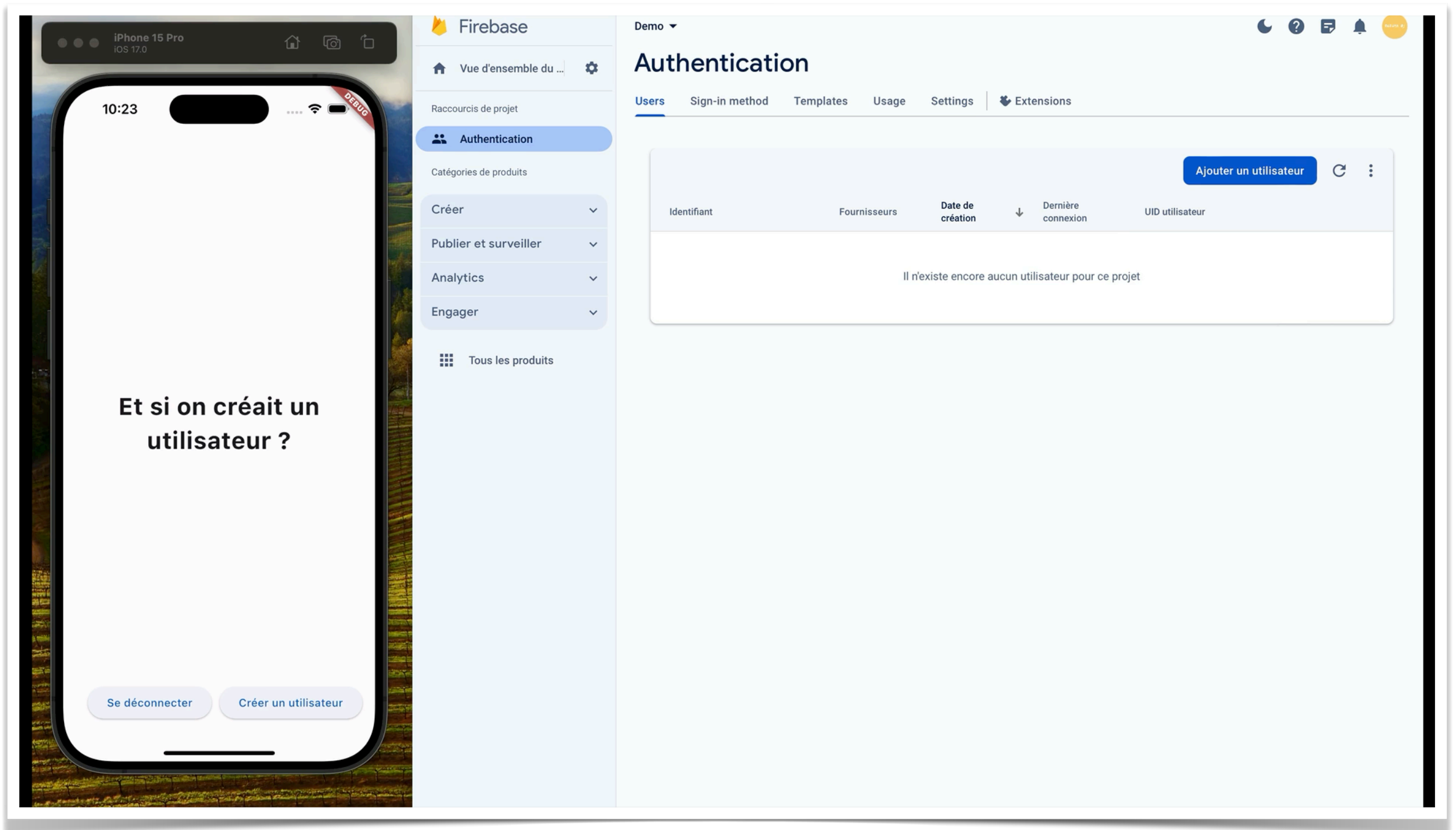


Firebase Authentication

Développement d'applications mobiles

Daniel Schreurs

Objectif



The image shows a mobile app interface on the left and the Firebase Authentication console on the right.

Mobile App Interface:

- Top status bar: iPhone 15 Pro, iOS 17.0, 10:23.
- Header: "Et si on créait un utilisateur ?"
- Buttons: "Se déconnecter" and "Créer un utilisateur".

Firebase Authentication Console:

- Header: "Authentication" with tabs: Users, Sign-in method, Templates, Usage, Settings, Extensions.
- Left sidebar: "Authentication" selected under "Raccourcis de projet".
- Main content area: "Ajouter un utilisateur" button and a table header.

Identifiant	Fournisseurs	Date de création	Dernière connexion	UID utilisateur
Il n'existe encore aucun utilisateur pour ce projet				

Sur l'organisation du cours

android	Initial commit	4 minutes ago
ios	Initial commit	4 minutes ago
lib	Initial commit	4 minutes ago
test	Initial commit	4 minutes ago
.gitignore	Initial commit	4 minutes ago
.metadata	Initial commit	4 minutes ago
README.md	Initial commit	4 minutes ago
analysis_options.yaml	Initial commit	4 minutes ago
pubspec.lock	Initial commit	4 minutes ago
pubspec.yaml	Initial commit	4 minutes ago

☰ README.md

Démo d'Authentification Firebase avec Flutter [↗](#)

Aperçu [↗](#)

Ce projet Flutter sert de démonstration simple de l'authentification Firebase en utilisant l'e-mail et le mot de passe. Il inclut des fonctionnalités de création d'utilisateur, de connexion et de déconnexion.

Prérequis [↗](#)

Avant d'exécuter l'application, assurez-vous d'avoir configuré Firebase pour votre projet. Mettez à jour le fichier

📖 Readme

📈 Activity

★ 0 stars

👁 2 watching

🔗 0 forks

Report repository

Releases

No releases published

[Create a new release](#)

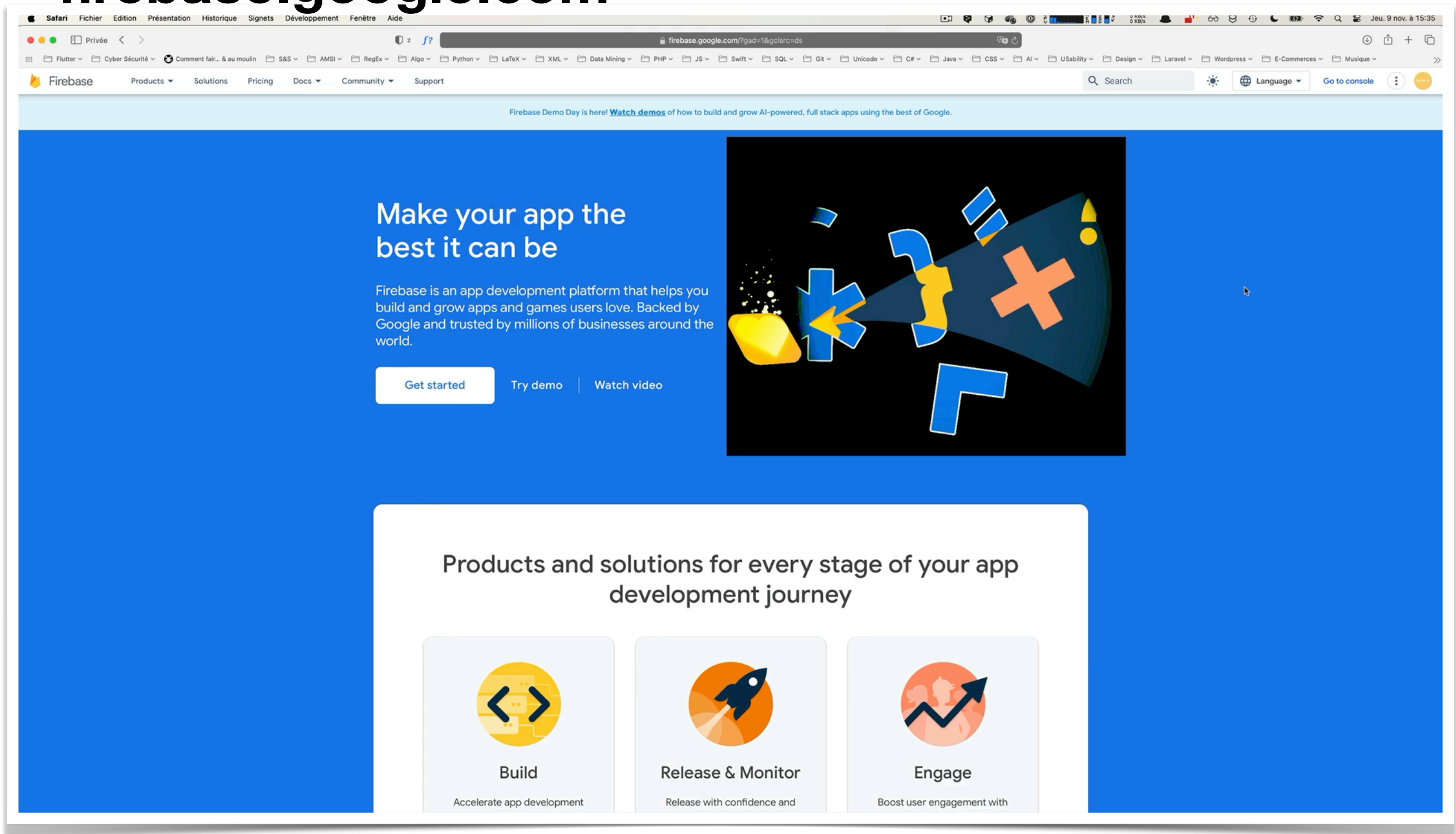
Packages

No packages published

[Publish your first package](#)



Activer le service firebase.google.com



The image shows a screenshot of the Firebase website homepage as seen in a Safari browser window. The browser's address bar shows the URL `firebase.google.com/?gad=1&gclid=ds`. The page features a blue header with the Firebase logo and navigation links: Products, Solutions, Pricing, Docs, Community, and Support. A search bar and a 'Go to console' link are also present. Below the header, a banner area with a blue background contains the text 'Make your app the best it can be' and a subtext: 'Firebase is an app development platform that helps you build and grow apps and games users love. Backed by Google and trusted by millions of businesses around the world.' Three buttons are visible: 'Get started', 'Try demo', and 'Watch video'. To the right of the text is an illustration of a game controller with a yellow arrow pointing towards a blue 'X' on a dark background. Below this banner, a white box titled 'Products and solutions for every stage of your app development journey' contains three cards: 'Build' (Accelerate app development) with a code icon, 'Release & Monitor' (Release with confidence and) with a rocket icon, and 'Engage' (Boost user engagement with) with a line graph icon.

Make your app the best it can be

Firebase is an app development platform that helps you build and grow apps and games users love. Backed by Google and trusted by millions of businesses around the world.

Get started Try demo Watch video

Products and solutions for every stage of your app development journey

Build
Accelerate app development

Release & Monitor
Release with confidence and

Engage
Boost user engagement with

Suivre ceci firebase.google.com

Tictic-2023

Formule Spark



Sélectionnez une plate-forme

× Ajouter Firebase à votre application Flutter

1 Préparer votre espace de travail

Le plus simple pour vous aider à démarrer est d'utiliser la CLI FlutterFire.

Avant de continuer, veuillez à :

- installer la [CLI Firebase](#) et à vous connecter (exécuter `firebase login`) ;
- installer le [SDK Flutter](#) ;
- créer un projet Flutter (exécuter `flutter create`)

Suivant

2 Installer et exécuter la CLI FlutterFire

Depuis n'importe quel répertoire, exécutez cette commande :

```
$ dart pub global activate flutterfire_cli
```



Ensuite, à la racine du répertoire de votre projet Flutter, exécutez cette commande :

```
$ flutterfire configure --project=tictic-2023
```



Cela enregistre automatiquement vos applications par plate-forme auprès de Firebase et ajoute un fichier de configuration `lib/firebase_options.dart` à votre projet Flutter.

Précédent

Suivant

Installer : node.js

nodejs.org 

Cette commande doit fonctionner



ABOUT | LEARN | DOWNLOAD | DOCS | GET INVOLVED | CERTIFICATION | NEWS

Downloads

Latest LTS Version: 20.9.0 (includes npm 10.1.0)

Download the Node.js source code or a pre-built installer for your platform, and start developing today.

LTS Recommended For Most Users	Current Latest Features	
 Windows Installer node-v20.9.0-x86.msi	 macOS Installer node-v20.9.0.pkg	 Source Code node-v20.9.0.tar.gz

Windows Installer (.msi)	32-bit	64-bit	ARM64
Windows Binary (.zip)	32-bit	64-bit	ARM64
macOS Installer (.pkg)	64-bit / ARM64		
macOS Binary (.tar.gz)	64-bit		ARM64
Linux Binaries (x64)	64-bit		
Linux Binaries (ARM)	ARMv7		ARMv8
Source Code	node-v20.9.0.tar.gz		

Additional Platforms

Docker Image	Official Node.js Docker Image
Linux on Power LE Systems	64-bit
Linux on System z	64-bit
AIX on Power Systems	64-bit

- Signed SHASUMS for release files (How to verify)
- All download options
- Installing Node.js via package manager
- Previous Releases
- Nightly builds
- Unofficial builds
- Building Node.js from source on supported platforms
- Installing Node.js via binary archive
- Install on Windows Subsystem for Linux (WSL)

Vous pouvez avoir une autre version.

Installer : CLI Firebase

binaire autonome npm

Pour utiliser `npm` (le gestionnaire de packages de nœuds) pour installer la CLI Firebase, procédez comme suit :

1. Installez [Node.js](#) à l'aide de [nvm-windows](#) (le gestionnaire de versions de Node). L'installation de Node.js installe automatiquement les outils de commande `npm`.

★ **Remarque** : La CLI Firebase nécessite **Node.js v16.13.0 ou version ultérieure**. Certaines fonctionnalités de Firebase peuvent nécessiter des versions spécifiques de Node.js. Consultez donc la page de démarrage de chaque produit Firebase pour connaître les exigences spécifiques de Node.js.

2. Installez la CLI Firebase via `npm` en exécutant la commande suivante :

```
$ npm install -g firebase-tools
```



Cette commande active la commande `firebase` disponible dans le monde entier.

★ **Remarque** : Si la commande `npm install -g firebase-tools` échoue, vous devrez peut-être [modifier les autorisations npm](#).

3. Continuez à [vous connecter et testez la CLI](#).



S'identifier dans votre terminal avec Google !

→ firebase login



Dans votre projet : ajouter les dépendances

2 Installer et exécuter la CLI FlutterFire

Depuis n'importe quel répertoire, exécutez cette commande :

```
$ dart pub global activate flutterfire_cli
```



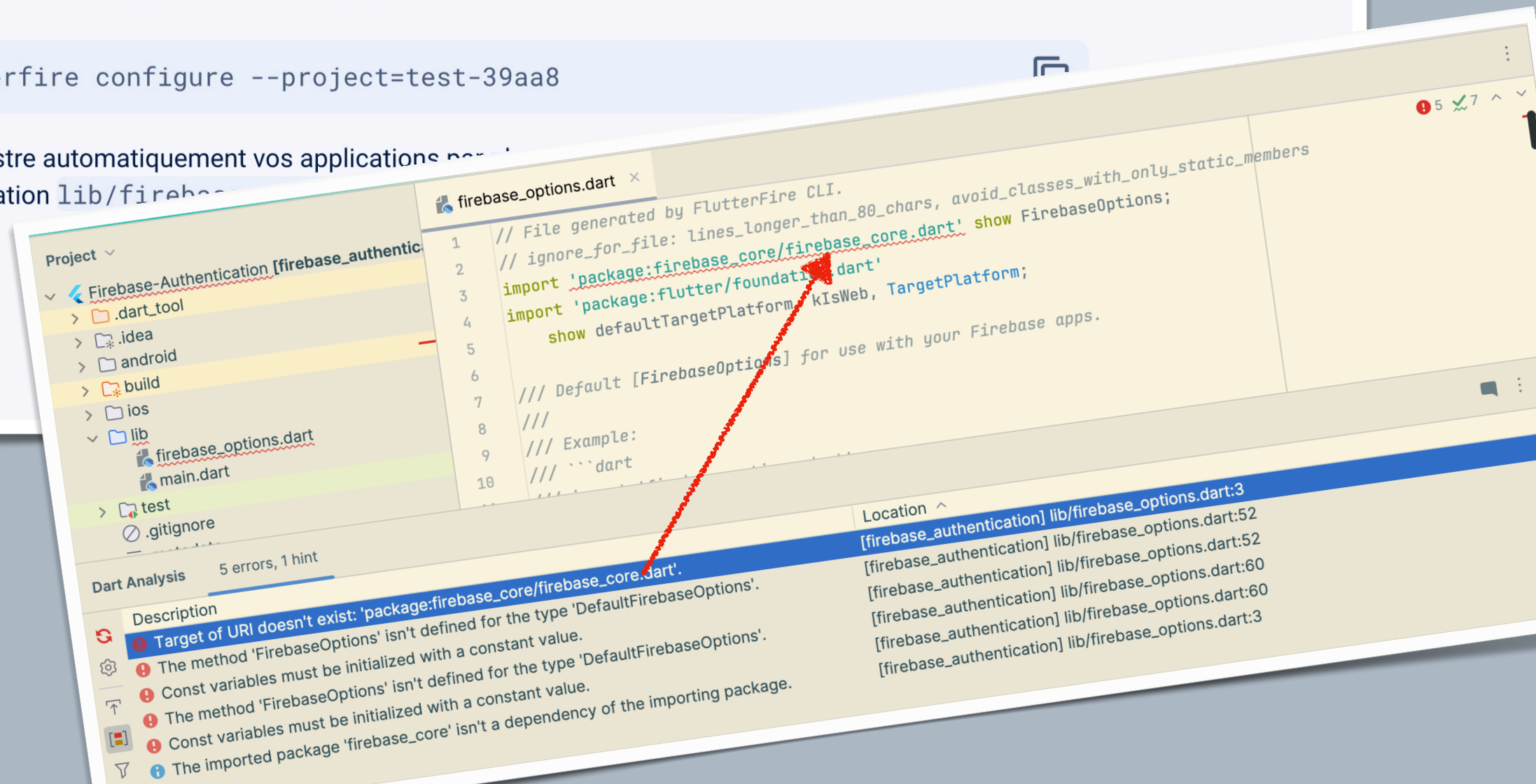
Ensuite, à la racine du répertoire de votre projet Flutter, exécutez cette commande :

```
$ flutterfire configure --project=test-39aa8
```

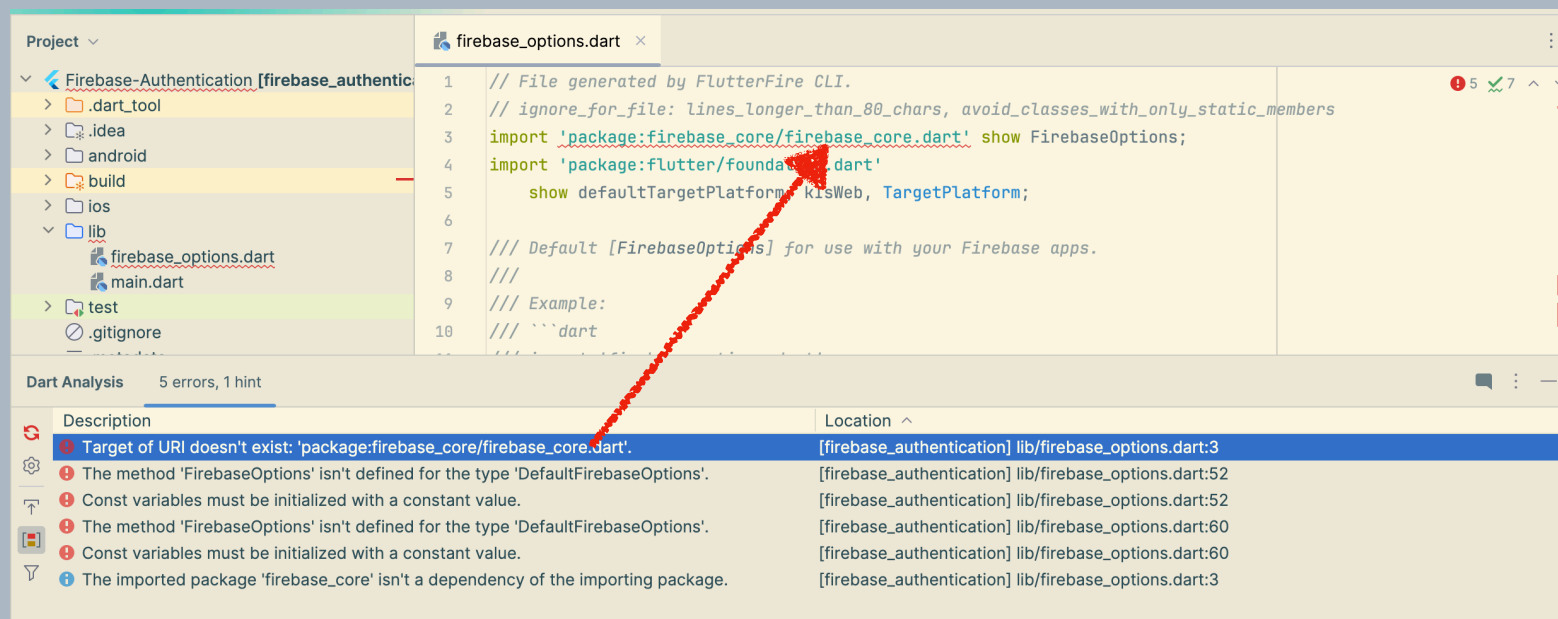


Cela enregistre automatiquement vos applications dans le fichier de configuration `lib/firebase_options.dart`.

Précédent



Dans votre projet : ajouter les dépendances



→ flutter pub add firebase_auth
→ flutter pub add firebase_core

Initialiser Firebase



```
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  runApp(const MyApp());  
}
```

Initialiser Firebase



```
Future<void> main() async {  
  WidgetsFlutterBinding.ensureInitialized();  
  await Firebase.initializeApp(  
    options: DefaultFirebaseOptions.currentPlatform,  
  );  
  runApp(const MyApp());  
}
```

Créer un widget class « StatefulWidget »

```
class CreateUserScreen extends StatefulWidget {
  const CreateUserScreen({super.key});

  @override
  State<CreateUserScreen> createState() => _CreateUserScreenState();
}

class _CreateUserScreenState extends State<CreateUserScreen> {
  final bool _userCreated = false;

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(mainAxisAlignment: MainAxisAlignment.center, children: [
        Center(
          child: Text(
            _userCreated
              ? "Bonjour \n${FirebaseAuth.instance.currentUser!.email}"
              : "Et si on créait un utilisateur ?",
            style: const TextStyle(fontSize: 30, fontWeight: FontWeight.bold),
            textAlign: TextAlign.center,
          ),
        ),
      ]),
      floatingActionButton: Row(
        mainAxisAlignment: MainAxisAlignment.end,
        children: [
          ElevatedButton(
            child: const Text("Se déconnecter"), onPressed: () async {}),
          const SizedBox(width: 10),
          ElevatedButton(
            child: const Text('Créer un utilisateur'),
            onPressed: () async {}),
        ],
      ),
    );
  }
}
```



Créer un utilisateur (synchrone)

Code bloquant

```
ElevatedButton(  
  child: const Text('Créer un utilisateur'),  
  onPressed: () async {  
    try {  
      await FirebaseAuth.instance  
        .createUserWithEmailAndPassword(  
          email: "${Random().nextInt(100000)}@gmail.com",  
          password: "1234567890");  
      setState(() {  
        _userCreated = true;  
      });  
    } on FirebaseAuthException catch (e) {  
      ScaffoldMessenger.of(context).showSnackBar(SnackBar(  
        showCloseIcon: true,  
        duration: const Duration(seconds: 10),  
        content: Text(e.message!),  
      ));  
    }  
  },  
)
```

Créer un utilisateur (asynchrone)

```
ElevatedButton(  
  child: const Text('Créer un utilisateur'),  
  onPressed: () async {  
    FirebaseAuth.instance  
      .createUserWithEmailAndPassword(  
        email: "${Random().nextInt(100000)}@gmail.com",  
        password: "1234567890")  
      .then((value) {  
        setState(() {  
          _userCreated = true;  
        });  
      }).onError((error, stackTrace) {  
        ScaffoldMessenger.of(context).showSnackBar(SnackBar(  
          showCloseIcon: true,  
          duration: const Duration(seconds: 10),  
          content: Text(error.toString()),  
        ));  
      });  
  },  
)
```

```
package:firebase_auth/firebase_auth.dart  
Future<UserCredential> createUserWithEmailAndPassword({required String email, required String password})
```

Déconnecter un utilisateur



```
ElevatedButton(  
  child: const Text("Se déconnecter"),  
  onPressed: () async {  
    await FirebaseAuth.instance.signOut();  
    setState(() {  
      _userCreated = false;  
    });  
  }),
```

Synthèse

- **Async** : Rendre asynchrone une fonction et donc non bloquante
- **await** : Attendre de résultats asynchrones
- **Future** : Représente une valeur disponible dans le futur. (Then, onError, etc.)
 - **Callback** : Fonction passée en paramètre, exécutée après la fin d'une opération asynchrone.
- **Stream** : Flux continu de données asynchrones

Sources

Sources et ressources à consulter

- [Get to know Cloud Firestore](#)
- [The Firebase Database For SQL Developers](#)
- [NoSQL data modeling](#)
- firebase.google.com
- [Firebase : Podcast](#)